

1. Understand Key-Value Storage Using Map (Interview Notes)

What is a Map?

A **Map** is a data structure that stores data in **key-value pairs**.

Each **key** is unique and maps to exactly one value.

```
Map students = new HashMap<>();

students.put(101, "Ali");
students.put(102, "Ahmed");
students.put(103, "Sara");
```

Here,

- **Key** → Student ID
 - **Value** → Student Name
-

Why Use a Map?

Use a Map when you need to:

- Store data using unique keys.
- Retrieve values quickly using a key.
- Represent real-world key-value relationships.

Examples:

- Student ID → Student
 - Username → Password
 - Product ID → Product
 - Country Code → Country Name
-

Real-world Example

```
Map marks = new HashMap<>();

marks.put("Ali", 90);
marks.put("Ahmed", 85);

System.out.println(marks.get("Ali"));
```

Output

90

Important Interview Points

- Map stores data as **key-value pairs**, not individual elements.
- **Keys must be unique**, but **values can be duplicated**.
- A key maps to **only one value**.
- Map is an **interface** and is **not part of the Collection hierarchy**.
- Common implementations are **HashMap**, **LinkedHashMap**, and **TreeMap**.
- Common methods:
 - put()
 - get()
 - remove()
 - containsKey()
 - containsValue()
- A Map cannot contain duplicate keys; inserting an existing key replaces its old value.

Tricky Interview Questions

Q1. Why isn't Map part of the Collection framework?

Answer:

Because Collection stores **individual elements**, whereas Map stores **key-value pairs**.

Q2. Can a Map have duplicate keys?

Answer: No.

Duplicate keys overwrite the previous value.

```
map.put(1, "Java");  
map.put(1, "Spring");
```

Result

```
1 → Spring
```

Q3. Can a Map have duplicate values?

Answer: Yes.

Multiple keys can point to the same value.

Q4. Can we iterate over a Map?

Answer: Yes.

Using:

- `keySet()`
 - `values()`
 - `entrySet()`
-

Q5. What is an Entry in a Map?

Answer:

A single **key-value pair**.

Example:

```
101 → Ali
```

Interview Summary

- **Map stores data as unique key-value pairs.**
- Keys are **unique**, while values **may be duplicated**.
- Map is **not a Collection** because it manages mappings instead of individual elements.
- Used for **fast retrieval** based on keys.
- Common implementations include **HashMap**, **LinkedHashMap**, and **TreeMap**.

Interview One-Liner:

"A Map stores data as unique key-value pairs, allowing fast retrieval of values using their associated keys."

2. Use HashMap for Fast Lookup by Key (Interview Notes)

What is HashMap?

HashMap is the most commonly used implementation of the **Map** interface.

It stores data as **key-value pairs** and provides **very fast lookup** using keys.

```
HashMap map = new HashMap<>();

map.put(101, "Ali");
map.put(102, "Ahmed");

System.out.println(map.get(101));
```

Output

```
Ali
```

Why Use HashMap?

Use HashMap when:

- Fast searching is required.
- Keys are unique.
- Element order is not important.

Examples:

- Login system
 - Student records
 - Product lookup
 - Caching
-

How Does HashMap Work?

Internally,

1. The key's **hashCode()** is calculated.
2. The hash code determines the **bucket**.
3. **equals()** is used (if needed) to find the correct key within that bucket.

Key

↓

hashCode()

↓

Bucket

↓

equals()

↓

Value

Important Interview Points

- HashMap implements the **Map** interface.
- Stores **key-value pairs**.

- Uses a **hash table** internally.
 - Lookup, insertion, and deletion are **$O(1)$ average**.
 - Does **not guarantee insertion order**.
 - Allows **one null key** and **multiple null values**.
 - Uses **hashCode()** and **equals()** for storing and retrieving keys.
 - Best choice when **fast key-based lookup** is more important than ordering.
-

Tricky Interview Questions

Q1. Why is HashMap so fast?

Answer:

Because it uses **hashing**, enabling **$O(1)$** average lookup.

Q2. Does HashMap maintain insertion order?

Answer: No.

Use LinkedHashMap if insertion order is required.

Q3. Can HashMap store duplicate keys?

Answer: No.

The new value replaces the old value.

Q4. Can HashMap store null?

Answer: Yes.

- **One null key**
 - **Multiple null values**
-

Q5. Which methods are used internally by HashMap?

Answer:

- `hashCode()`
 - `equals()`
-

Q6. What happens if two keys have the same hash code?

Answer:

This is called a **hash collision**.

HashMap uses `equals()` to distinguish between the keys.

Q7. What is the average time complexity of `get()` in HashMap?

Answer:

$O(1)$ average.

Worst case can be **$O(n)$** (or **$O(\log n)$** in modern Java implementations after treeification), but interviews generally expect **$O(1)$ average**.

Interview Summary

- **HashMap stores unique key-value pairs using a hash table.**
- Provides **$O(1)$ average** time complexity for `put()`, `get()`, and `remove()`.
- Does **not preserve insertion order**.
- Allows **one null key** and **multiple null values**.
- Relies on **`hashCode()` and `equals()`** for efficient key lookup.
- Preferred when **fast retrieval by key** is the primary requirement.

Interview One-Liner:

"HashMap uses hashing to provide **$O(1)$ average-time key-based lookup**, making it the preferred choice for fast key-value storage when ordering is not required."